

Perfeito — fiz uma análise completa do código atual e abaixo está o relatório técnico detalhado que você pediu.

Escopo analisado

Analisei os fluxos reais implementados em:

- Scanner de datasets.
 - Importador principal.
 - Organizador de pares.
 - Normalização/tokenização.
 - Motor de tradução (busca, cache, reconstrução, pós-processamento).
 - Banco/índices/deduplicação.
-

Fluxo completo da importação

1) Scanner

- DatasetScanner::scan() varre datasets/ recursivamente e agrupa candidatos por corpus+par de idioma.
- Só considera arquivo legível, com tamanho mínimo, nome válido, idioma reconhecido e conteúdo inicial não vazio.
- Faz validação de coerência de par por razão de tamanho (máx. 10x).

2) Parser

- O “parser” aqui é estrutural (nome/extensão/lang pair), não parsing linguístico profundo:
 - Extrai idioma da extensão (.en, .pt_BR, etc).
 - Extrai source-target do basename e separa pelo -.

3) Filtros

- Na importação linha-a-linha (importMosesDataset), cada linha passa por:
 - cleanText (normalização de whitespace).
 - isValidTranslationPair com filtros de ruído técnico (HTML/CSS/URL/código/tokens etc.).
 - filtro de tamanho mínimo/máximo e igualdade normalizada.
 - **novo limite de 8 palavras** (descarta acima disso).

4) Normalização

- Idiomas normalizados em LanguageNormalizer antes de processar/importar.
- source_text salvo no banco em forma normalizada (normalizedSourceText).
- target_text é salvo como veio do organizer (sem normalização agressiva igual source).

5) Deduplicação

- Durante insert usa ON CONFLICT(source_text, translated_text, source_lang, target_lang) DO UPDATE frequency = frequency + 1.
- Pós-importação roda cleanupDatabase(), removendo inválidas e consolidando duplicatas por agrupamento.

6) Banco de dados

- Tabela translations com colunas source_text, translated_text, source_lang, target_lang, frequency.
- Tabela import_progress para retomada de importação.

7) Índices

- Índices simples + índice composto de lookup + índice único do par completo.

8) Organizer

- TranslationOrganizer::organizePair atualmente é **pass-through** (não reordena, não alinha, não infere).

9) Pós-processamento

- cleanupDatabase() faz saneamento e otimização final após importação.
-

Fluxo completo da tradução

1) Tokenização

- Segmentação em palavras via TextNormalizer::words (normaliza e split por whitespace).
- Extração de tokens “originais” (com regex de palavra) via sourceWordTokens.

2) Normalização

- normalizeForLookup:
 - lowercase,
 - padroniza aspas,
 - expande contrações,
 - remove pontuação/símbolos para lookup.
- Esse texto normalizado é a chave de busca no banco.

3) Busca

- findBestMatch tenta janelas decrescentes (máximo 8 palavras, depois 7...1).
- Para cada slice gera candidatos (exact + compact, removendo palavras opcionais como the, please).
- Consulta lote no banco por source_lang, target_lang, source_text.

4) Seleção de correspondência

- Banco retorna por frequência desc e menor tradução no desempate (ORDER BY ... frequency DESC, LENGTH(translated_text) ASC).
- findBestMatch prioriza maior n-grama encontrado.

5) Cache

- Cache em memória (QueryCache) por chave source|sourceLang|targetLang.

6) Reconstrução

- Se match completo do segmento: aplica separadores do source (applySourceSeparators).
- Se match parcial: monta translatedParts e preserva token original onde não houver match.

7) Pós-processamento

- Reaplica pontuação inline (reapplyInlinePunctuation).
 - Ajusta capitalização (applyCapitalization).
 - Reanexa delimitadores/whitespace por segmento/linha.
-

Locais que mexem com corte/divisão/fragmentos/alinhamento/pontuação/reorganização

A) Cortam frases

1. Tokenizer::splitSentences
 - **Arquivo/Função:** core/Tokenizer.cpp / splitSentences.
 - **Motivo:** separar por fim de frase ([.!?]).
 - **Impacto atual:** pode quebrar abreviações se usado diretamente.
 - **Correção recomendada:** manter apenas para utilitários; no fluxo principal já existe segmentação mais robusta no engine.

2. TranslatorEngine::findBestMatch (janela decrescente)

- **Arquivo/Função:** core/TranslatorEngine.cpp / findBestMatch.
- **Motivo:** buscar maior frase possível.
- **Impacto atual:** fragmenta em n-gramas quando não encontra match longo.
- **Correção recomendada:** registrar “queda de tamanho” em log/telemetria para diagnóstico de dataset ruim.

B) Dividem palavras

1. TextNormalizer::words

- **Arquivo/Função:** core/TextNormalizer.cpp / words.
- **Motivo:** tokenizar para lookup.
- **Impacto:** pontuação vira separador, contrações são expandidas antes.
- **Correção:** se quiser preservar compostos específicos, criar whitelist de padrões.

2. sourceWordTokens

- **Arquivo/Função:** core/TranslatorEngine.cpp / helper local.
- **Motivo:** reaproveitar tokens originais para fallback.
- **Impacto:** diferença entre tokenização “lookup” e “display” pode gerar pequenas divergências.

C) Reaproveitam fragmentos

1. Fallback por palavra original sem tradução

- **Arquivo/Função:** translateSegment.
- **Motivo:** não inventar tradução.
- **Impacto:** saída mista (parte traduzida, parte original).
- **Correção:** opcional: marcar visualmente trechos não traduzidos.

2. lookupCandidates com versão “compact”

- **Arquivo/Função:** core/TextNormalizer.cpp / lookupCandidates.
- **Motivo:** tolerar artigos/palavras opcionais.
- **Impacto:** aumenta recall, mas pode introduzir match semântico mais frouxo.

D) Tentam inferir alinhamentos

1. applySourceSeparators

- **Arquivo/Função:** core/TranslatorEngine.cpp / applySourceSeparators.
- **Motivo:** recolocar separadores do source entre palavras traduzidas.
- **Impacto:** **alinhamento implícito posicional 1:1** só ocorre se contagem de palavras bater.
- **Correção:** usar alinhamento por spans/tags de placeholder em vez de posição.

E) Modificam pontuação

1. cleanStoredTranslationForSource

- **Arquivo/Função:** core/TranslatorEngine.cpp.
- **Motivo:** sanitizar ruído de pontuação do dataset.
- **Impacto:** substitui símbolos por espaço (pode apagar pontuação útil).

2. reapplyInlinePunctuation + inlinePunctuationMarks

- **Arquivo/Função:** core/TranslatorEngine.cpp.
- **Motivo:** manter vírgula/;/: do source.
- **Impacto:** reaplica por índice de palavra, suscetível a reordenação linguística.

F) Reorganizam traduções

1. TranslationOrganizer::organizePair

- **Arquivo/Função:** core/TranslationOrganizer.cpp.
 - **Motivo:** ponto único para regras futuras.
 - **Impacto atual:** não reorganiza nada (pass-through).
-

Respostas explícitas (sim/não)

1. **Existe qualquer alinhamento implícito entre palavras?**
SIM. Em applySourceSeparators e reaplicação de pontuação, quando assume correspondência por posição/token count.
 2. **Existe qualquer geração automática de pares derivados?**
NÃO no Organizer atual. O organizer não cria pares derivados (pass-through).
SIM em sentido de normalização de chave de lookup, pois source_text é normalizado antes de salvar.
 3. **Existe qualquer fragmentação durante importação?**
NÃO fragmenta em subfrases para gravar. Importa linha inteira (se válida) ou descarta.
SIM há divisão para contagem/filtro (split("\\s+")) para aplicar limite de 8 palavras.
 4. **Existe qualquer fragmentação durante tradução?**
SIM. O motor tenta do maior n-grama ao menor e compõe a saída por partes quando não acha match completo.
-

Diagnóstico objetivo do seu problema (“troca red=dragão / dragon=vermelho”)

- O motor **tenta frase completa primeiro**; então o erro típico vem de:
 1. par errado/ruim no banco (dataset desalinhado localmente), ou
 2. fallback por fragmentos quando não encontra a frase completa.
 - O risco é aumentado pelos candidatos compactados (lookupCandidates) e por pós-processamento posicional (separadores/pontuação).
-

Correção recomendada (prioridade alta)

1. **Adicionar “modo estrito de frase” para 2+ palavras**
 - Se segmento tiver ≥ 2 palavras e não houver match completo, **não fragmentar para palavra** automaticamente (ou fragmentar só até bigrama).
2. **Telemetria completa por segmento**
 - logar: texto, janelas tentadas, candidato escolhido, origem (cache/db), fallback usado.
3. **Validação de alinhamento no import**
 - heurística simples: diferença de comprimento em palavras, proporção, stopwords extremas, rejeição de par suspeito (além dos filtros atuais).
4. **Desativar/parametrizar lookupCandidates compacto**
 - para evitar “match frouxo” em produção.

Se você quiser, no próximo passo eu implemento isso diretamente no código e te entrego com relatório de execução linha por linha.